CS 331 (Software Engineering Lab)

Assignment 4

CI/CD Pipeline Automated Failure Recovery System

Part I: Software Architecture Style

Selected Architecture: Layered Architecture (N-Tier)

For the CI/CD Automated Failure Recovery System, I have chosen the Layered Architecture (also known as N-Tier Architecture) pattern. This architecture organizes the system into horizontal layers, where each layer has a specific role and communicates only with adjacent layers.

System Architecture Structure:

PRESENTATION LAYER

- **Web Dashboard (React/Vue.js)**
- **REST API Endpoints**
- **CLI Interface**
- **Notification Handlers**

BUSINESS LOGIC LAYER

- **Pipeline Monitor Service**
- **Failure Detection Engine**
- **Root Cause Analyzer**
- **Recovery Engine**
- **Learning Module**

DATA ACCESS LAYER

- **Repository Pattern**
- **ORM (Object-Relational Mapping)**
- **Cache Manager**
- **External API Connectors**

DATA STORAGE LAYER

- PostgreSQL Database (Primary)
- Redis Cache
- MongoDB (Logs & Metrics)
- File Storage (ML Models)

EXTERNAL INTEGRATION LAYER

- CI/CD Platform APIs (Jenkins, GitLab, GitHub)
- Version Control Systems (Git)
- Notification Services (Email, Slack, Teams)
- Monitoring Tools (Prometheus, Grafana)

Part I-A: Justification - Component Granularity

The CI/CD Automated Failure Recovery System follows the Layered Architecture pattern based on the following component granularity and characteristics:

1. Presentation Layer (User Interface)

Components:

- Web Dashboard - React-based UI for monitoring and configuration
- REST API Gateway - Express.js/FastAPI endpoints for external access
- CLI Tool - Command-line interface for automation scripts
- Notification Manager - Handles alerts via Email, Slack, Teams

Granularity Justification:

- Coarse-grained components focusing on user interaction
- Each component handles one specific presentation concern
- Stateless design - no business logic in this layer
- Clear separation between UI rendering and data processing

2. Business Logic Layer (Core Engine)

Components:

- Pipeline Monitor Service - Continuously monitors pipeline executions
- Failure Detection Engine - Identifies and classifies failures
- Root Cause Analyzer - Diagnoses underlying causes using ML
- Recovery Engine - Orchestrates recovery strategy execution
- Strategy Manager - Manages recovery strategy selection
- Learning Module - Trains ML models from historical data
- Policy Manager - Manages recovery rules and thresholds

Granularity Justification:

- Medium-grained components - each handles one business capability
- Components are cohesive - single responsibility principle
- Loosely coupled through interfaces and dependency injection
- Stateful but maintains state in data layer, not in-memory
- Domain-driven design with clear boundaries

3. Data Access Layer (Persistence)

Components:

- Pipeline Repository - CRUD operations for pipeline data
- Failure Repository - Manages failure records and history
- Recovery Repository - Stores recovery attempts and results
- Configuration Repository - Manages system settings
- Cache Manager - Redis-based caching for performance
- API Connector Pool - Manages external API connections

Granularity Justification:

- Fine-grained data access components
- Repository pattern abstracts data storage implementation
- Each repository handles one entity type

○ No business logic - pure data access operations

○ Supports multiple data sources transparently

4. Data Storage Layer

Components:

• PostgreSQL - Relational data (pipelines, configurations, users)

• MongoDB - Document store for logs and unstructured metrics

• Redis - In-memory cache for hot data and session management

• File System - ML model storage and archived logs

Granularity Justification:

○ Infrastructure-level components

○ Each database serves specific data characteristics

○ Polyglot persistence based on data access patterns

○ Horizontal scalability through database-specific features

5. External Integration Layer

Components:

• CI/CD Platform Adapters - Jenkins, GitLab CI, GitHub Actions connectors

• Version Control Adapters - Git API integration

• Notification Service Adapters - Email, Slack, MS Teams integration

• Monitoring Service Adapters - Prometheus, Grafana connectors

Granularity Justification:

○ Adapter pattern for external service integration

○ Each adapter encapsulates one external platform

○ Provides uniform interface regardless of underlying platform

○ Easy to add new platform support without changing core system

Summary: Why This is Layered Architecture

Clear horizontal separation of concerns into 5 distinct layers

Each layer has well-defined responsibilities and boundaries

Strict layer communication - each layer only communicates with adjacent layers

Lower layers provide services to higher layers

Higher layers depend on abstractions, not concrete implementations

Components within each layer have similar granularity and cohesion

No circular dependencies between layers